CHRONOGUARD

System Architecture & Flowcharts

Self-Healing Dependency Guardian

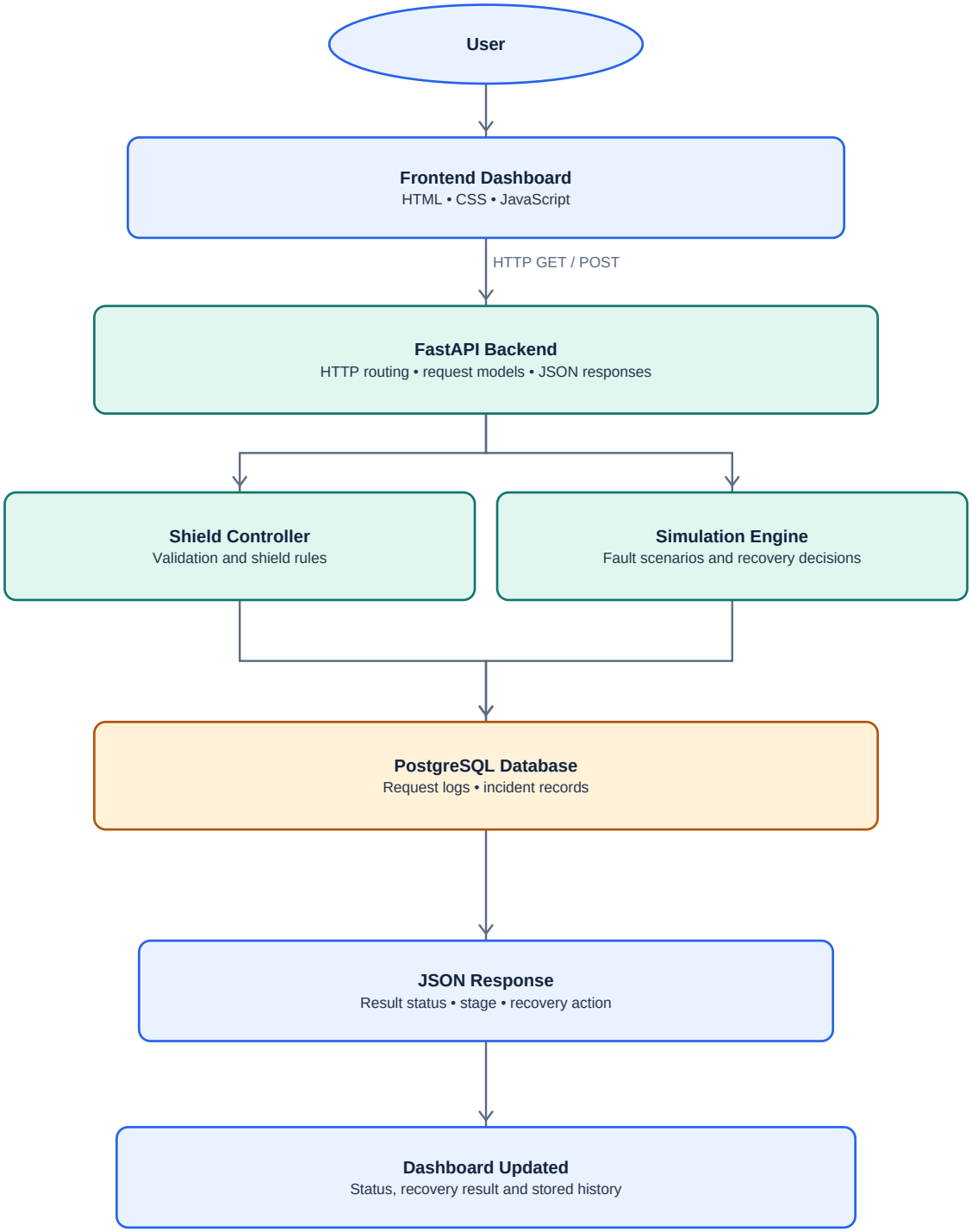
UPDATED TECHNICAL DIAGRAMS

Overall architecture • server handling • fault recovery • database logging • reload • deployment

Prepared from the architecture and flowcharts presented in this chat

1. Overall System Architecture

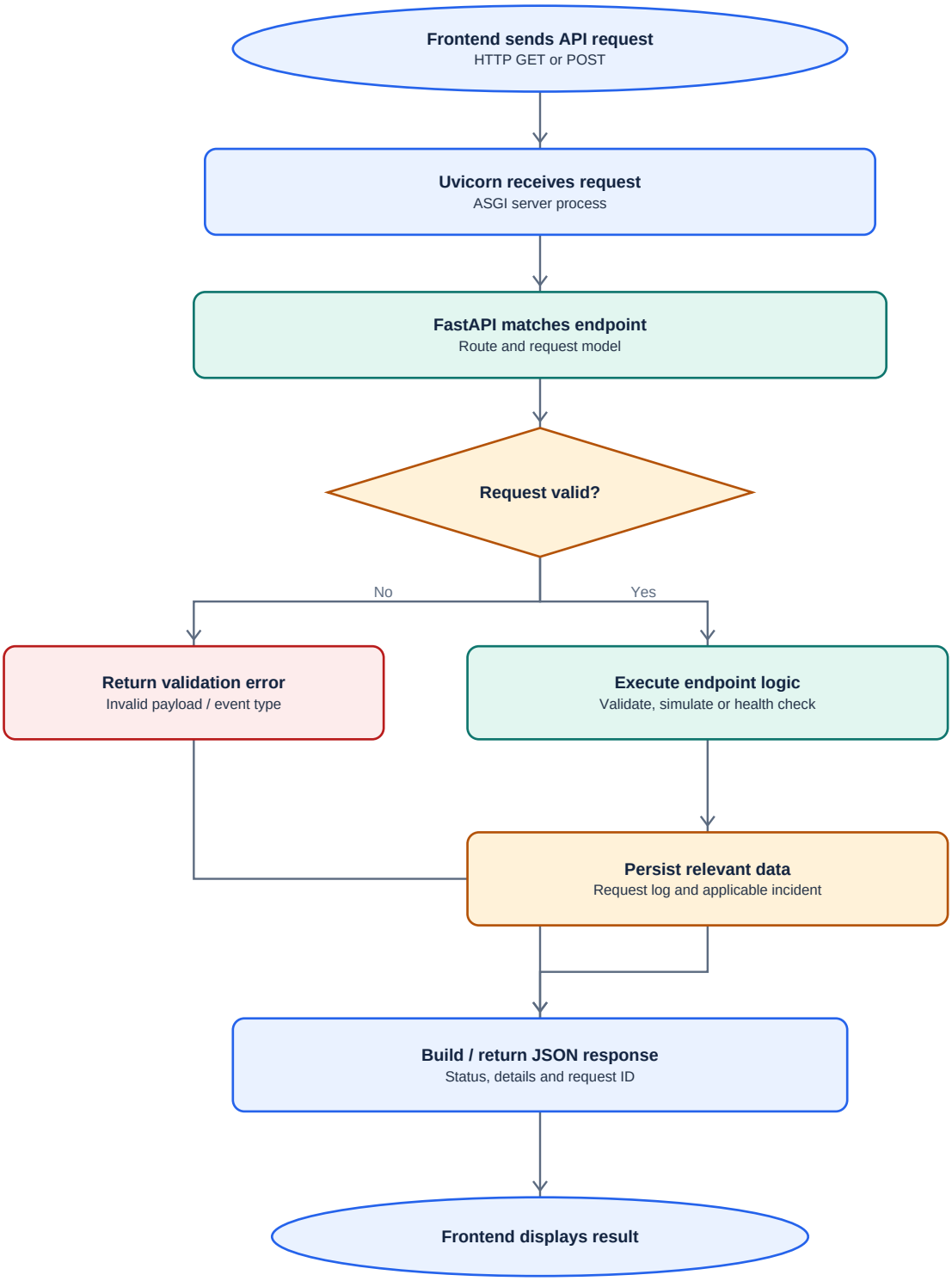
ChronoGuard connects the browser dashboard to a FastAPI backend and PostgreSQL. The backend handles validation and simulation logic, while the database stores request history and applicable incident records. The frontend displays the response and refreshes its log table.



Key: the frontend initiates actions; FastAPI processes them; PostgreSQL persists the relevant results. The simulation engine's backup-path decision is currently application-level simulation, not proof of a live backup server.

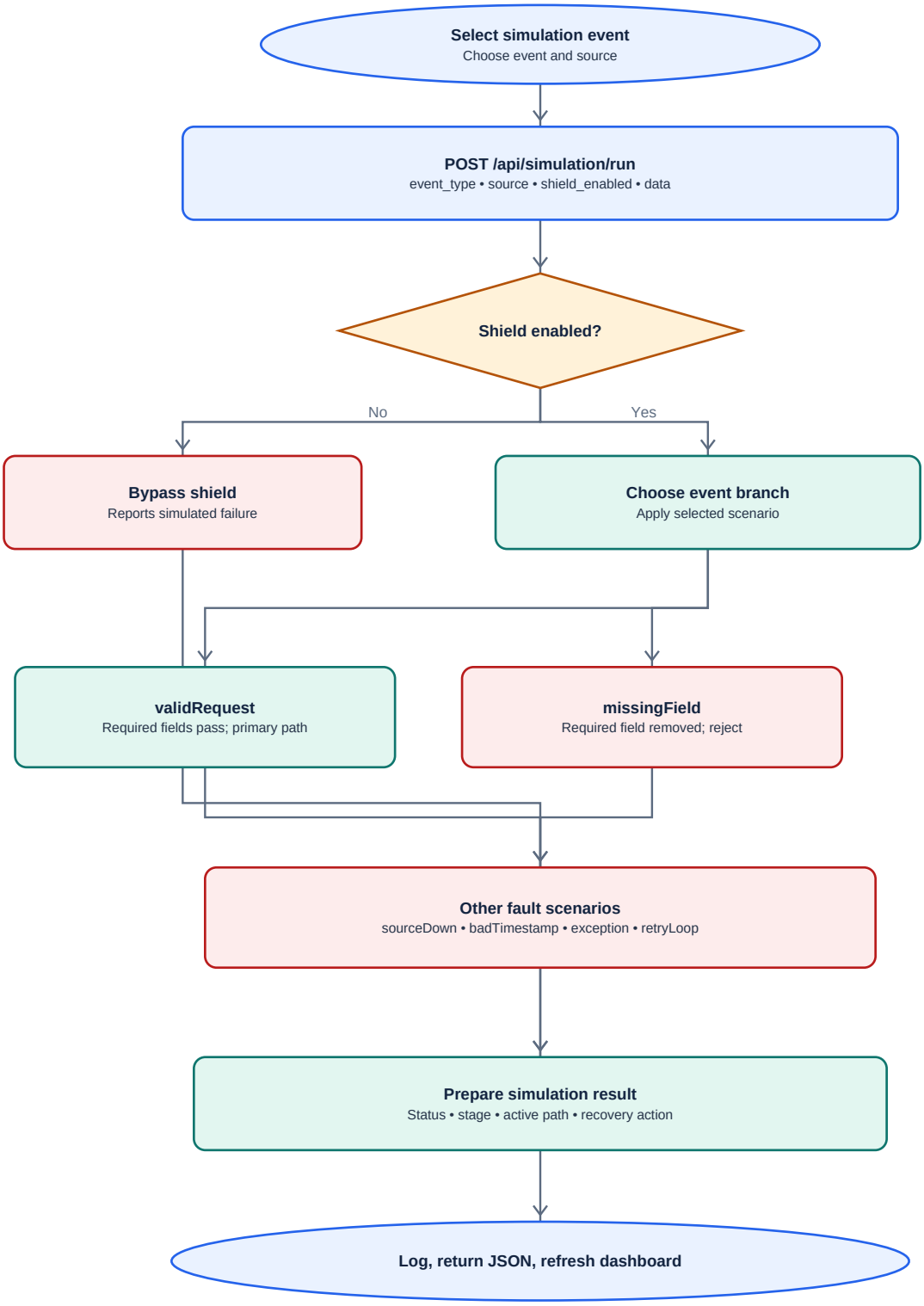
2. Server Request-Handling Flowchart

This diagram shows the common lifecycle of an API call. Exact behavior depends on the endpoint: validation endpoints check fields, the simulation endpoint runs an event scenario, and the health endpoint checks service/database availability.



3. Fault Detection and Recovery Flowchart

The user selects an event type, the backend runs its corresponding scenario, and the response is stored and returned. Recovery labels describe the configured simulation outcome; they do not imply that live traffic has moved to a separate physical server.



4. Event Types and Expected Outcomes

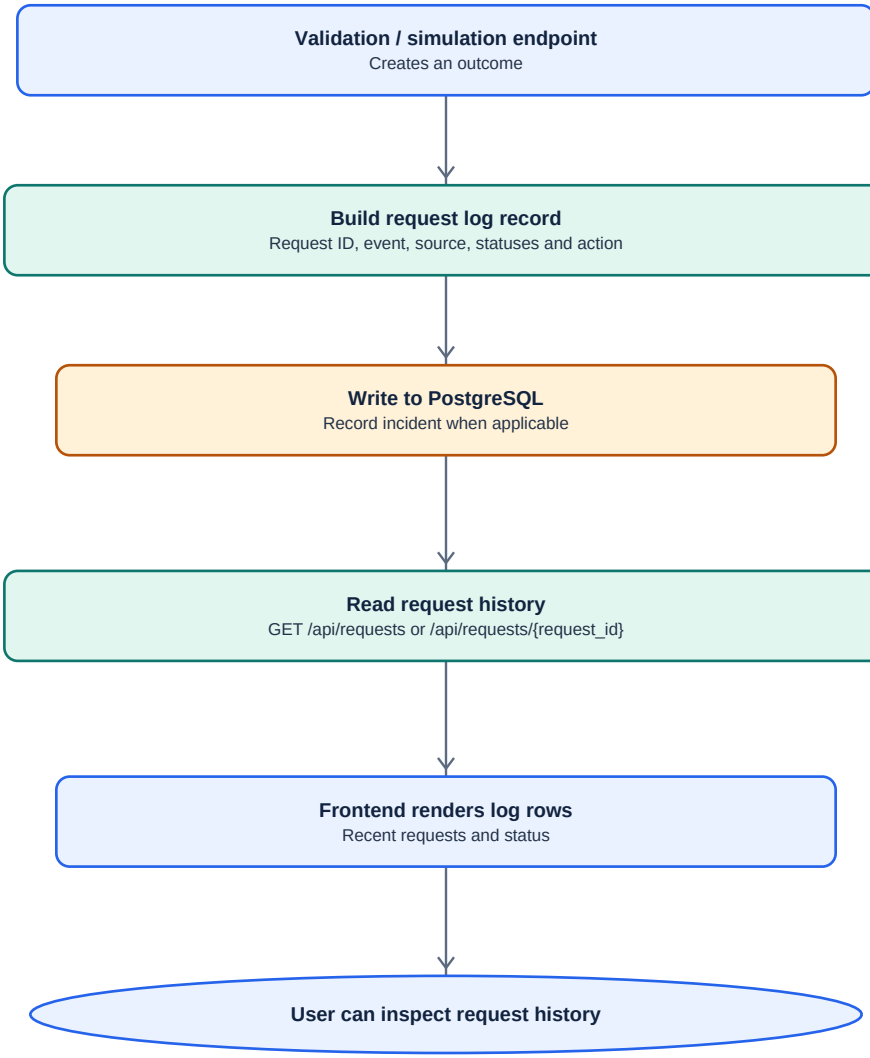
These are the event behaviors represented by the current simulation flow. The exact response should be verified against the API output and stored request record during testing.

Event type	What the backend simulation does	Typical reported outcome
validRequest	Checks required fields and processes on the primary path.	validation passed; processed
missingField	Removes a required field for validation; rejects when a required field is missing.	rejected
sourceDown	Selects an alternate primary/backup path in the scenario.	recovered
badTimestamp	Runs the configured timestamp-fault scenario and reports backup recovery.	recovered
exception	Runs the configured exception scenario and reports backup recovery.	recovered
retryLoop	Runs the configured retry-limit scenario and reports a limited/recovered result.	limited / recovered

Interpretation guide

- **validation_status** describes the validation result or bypass/limit state.
- **final_status** describes the simulation's final reported result.
- **processing_stage** identifies the named stage in the simulated workflow.
- **active_server** is the path/server label selected by the simulation; it does not by itself verify a real server's health.

5. Database Logging Flowchart

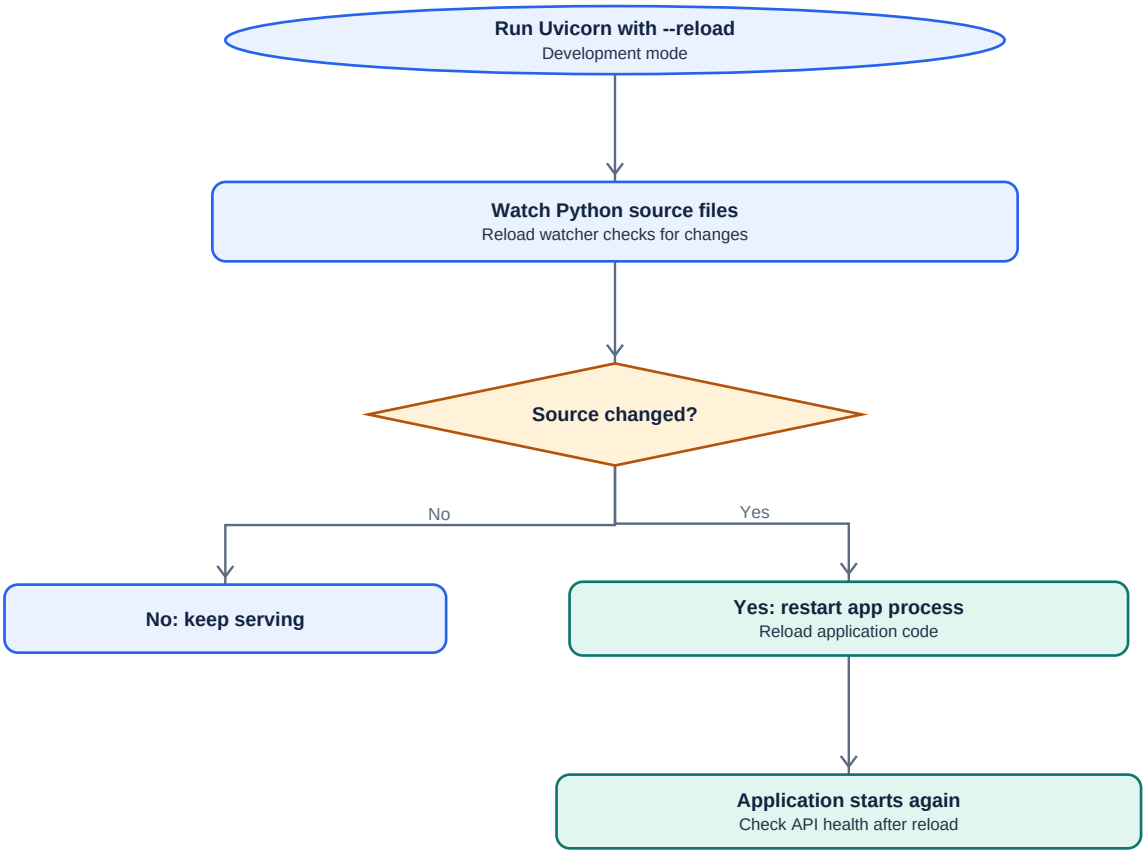

Validation / simulation endpoint
Creates an outcome
Build request log record
Request ID, event, source, statuses and action
Write to PostgreSQL
Record incident when applicable
Read request history
GET /api/requests or /api/requests/{request_id}
Frontend renders log rows
Recent requests and status
User can inspect request history

6. API Endpoint Reference

Endpoint	Purpose
GET /	Basic API root response.
GET /api/health	Checks API health and PostgreSQL connectivity.
POST /api/shield/validate-request	Validates a request using shield rules.
POST /api/shield/validate-age	Runs age-validation logic.
POST /api/simulation/run	Runs a valid-request or fault simulation.
GET /api/requests	Returns stored request history.
GET /api/requests/{request_id}	Fetches a stored request by request ID.

7. Server Reloading vs. Fault Recovery

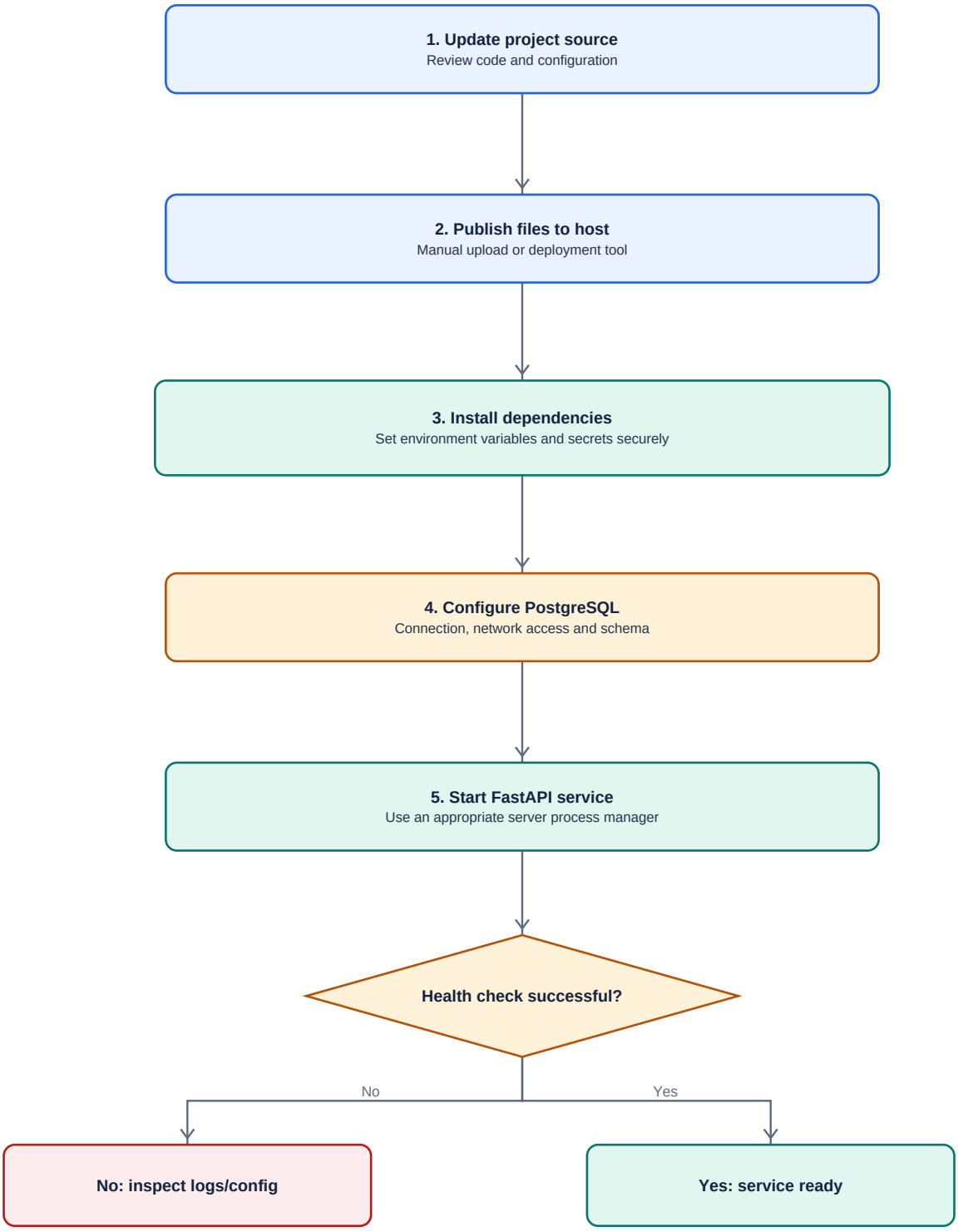
Mechanism	Trigger	Behavior	Current status
Uvicorn development reload	Python source file changes while --reload is enabled.	Restarts the development application process and imports code again.	Used in local development.
Simulated recovery	A user submits a fault scenario.	Application logic returns a recovery action and a selected path label.	Implemented as simulation.
Production failover	A real instance or dependency becomes unhealthy.	Health monitoring and traffic routing move work to a healthy instance.	Not demonstrated as live infrastructure.
Upload / deployment	Code is published to a host or deployment service.	Files and dependencies are prepared, configuration applied, then service starts.	No verified automated pipeline established.



Important: `uvicorn app.main:app --reload` is for development. It is not a load balancer, a guarantee of recovery from every runtime failure, or production high availability.

8. Upload / Deployment Workflow

This is a general deployment workflow for the project, not a claim that an automated deployment pipeline already exists.



Recommended post-deployment checks

1. Confirm the service starts without import/configuration errors. 2. Call GET /api/health. 3. Verify the database connection. 4. Run a valid-request test and a fault-simulation test. 5. Confirm GET /api/requests returns the new request record.

9. Testing Checklist

Test	Expected check	Evidence to capture
API health	API reports healthy and database connected.	GET /api/health response.
Valid request	Required fields pass; final status is processed.	Simulation response and request history row.
Missing field	Scenario rejects missing required field.	Response status and saved log.
Source down	Alternate path is selected by the simulation.	active_server and recovery_action.
Bad timestamp	Configured timestamp scenario reports expected outcome.	Response and stored request row.
Exception	Configured exception scenario reports recovery.	Response and stored request row.
Retry loop	Retry-limit scenario reports limited/recovered result.	Response and stored request row.
Development reload	Source edit triggers reload when --reload is enabled.	Uvicorn console and successful health check.
Database history	Request-list endpoint returns stored rows.	JSON result with request ID and event type.

10. Limitations and Future Enhancements

Current limitation	Suggested enhancement
Backup path is selected in simulation logic.	Deploy independent instances and add real health-based traffic failover.
Uvicorn reload is a development convenience.	Use production server management, container orchestration or equivalent deployment controls.
Frontend may keep local simulated graph state.	Prefer rendering actual backend response data for authoritative status.
Payload logging can expose sensitive data if filtering is incomplete.	Apply consistent redaction, minimize stored data, and avoid logging credentials.
No verified automated upload pipeline.	Add CI/CD, automated tests, environment configuration and rollback.
Health check is a limited connectivity test.	Add readiness/liveness checks and dependency monitoring if needed.

11. Project Summary

ChronoGuard links a frontend dashboard to a FastAPI backend and PostgreSQL. The server routes and validates requests, runs the selected fault or valid-request simulation, prepares status and recovery details, stores request history and applicable incidents, and returns JSON. Uvicorn `--reload` restarts the development process when watched source files change. Real multi-server failover and automated production deployment are future enhancements rather than demonstrated capabilities.

Suggested presentation statement

“ChronoGuard demonstrates request validation, fault simulation, recovery decisions, and database logging. FastAPI handles API requests, Uvicorn runs the application, and PostgreSQL stores operational history. The current recovery behavior is simulated; real high availability and production deployment would require additional infrastructure.”